From classifier to product: a self-explaining LULC workbench, a richer model zoo, and portable outputs

Salil Gujar
M.Tech CSE Entry No. 2025MCS2106

July 2026

Where we are, and what week 8 tackled

Done so far

- A 4-class base map over India at 10 m with a living hierarchy: split or add a class, give a few example polygons, and retrain that node on the fly.
- A git-backed zoo of Model and Dataset cards, with save, reload, merge, base and year selection, publishing, and site testing on acacia, tea, and mining.

What was advised (26 points)

- Make it explainable and controllable: show only the controls that fit the flow, and guide the user through the pipeline.
- Make the zoo honest: standard classes, dataset-to-model links, selective publishing.
- Add new abilities: Tessera training, an auto model bake-off, a GeoTIFF export, a resumable project.
- Answer the operational questions on schemas, deployment, and running Tessera with joblib.

A workbench that explains itself

- Two panels, one job each. The left panel is navigation (area, hierarchy, save, zoo); a new right panel is contextual: click a class and it shows exactly what you can do with it, in order: mark data, split, add, merge, retrain.
- Only the relevant controls. The balancing and multi-year knobs appear only when you are training your own split, not for a bare leaf or the base map.
- Two ways to see the scheme. “By hierarchy” is the class tree; “by operations” is the ordered sequence that built it, and clicking a step acts on that class. The same right panel serves both views.
- Draw the area on the map, with a rectangle tool or a click-to-centre size slider; the classification is clipped to the box, and an eye toggle hides or shows it.
- Moving to a new area offers a clean reset, confirmed first, so a scheme built for one site does not leak into the next.

Model zoo improvements

- Every trained model shows up as a card. We fixed the case where re-splitting a node dropped its old model, so nothing trained goes missing, and any card can be deleted.
- Standard classes on the card. If the uploader maps a class to a standard scheme (ESA WorldCover or USDA), the small card shows the standard name, for example “Tree cover”; the detail pane still shows what the uploader calls it, acacia mapped to Tree cover.
- Provenance both ways. A dataset card now lists which models use it, alongside the existing model-to-dataset links, so one dataset feeding several models is visible from either side.
- Selective publishing. Tick the cards you want and publish those, instead of only “publish all”.

Auto model bake-off

"Auto" trains several models on the same held-out-polygon split and keeps the most accurate. We restrict the field to linear models, because only a linear model replays as Earth-Engine band math, so the winner still renders as the crisp 10 m tile map.

Candidate (acacia / non-acacia)	Held-out accuracy
LinearSVC	0.703
Logistic Regression	0.692
Ridge	0.731

Tessera as a training embedding

- A user can now train a split on Tessera (128-d) instead of Alpha Earth (64-d), for example acacia against non-acacia. It is scored, carded, and kept like any other model.
- On a Delhi run, acacia against non-acacia on Tessera reached about 0.73 held-out, comparable to Alpha Earth on the same split.
- Our four requested ROIs were approved for 2017 to 2025 (v1.1, cambridge variant). Because only these four boxes were approved, we have the multi-year Tessera data only inside those boxes, so Tessera training is offered only there.
- A Tessera split is a per-point download rather than an Earth-Engine image, so it does not ride the crisp tile map. It is evaluated and carded, and the card is marked accordingly.

Portable outputs: GeoTIFF and a resumable project

- Download the result as a GeoTIFF. The classified area exports as a single-band integer-class raster at 10 m, with a code-to-class legend, built server-side and fetched by the browser, ready to open in QGIS.
- Save the whole project as one JSON: the class scheme, the sequence of steps that built it, and the area, year, and base classes. Datasets and models ride as links, never copied. Reloading resumes the exact view.

New ground truth: seasonal water

We downloaded a set of water and non-water polygons for farm ponds and water bodies, marked across about ten dates each between 2019 and 2025, and folded them into the zoo as a dataset card. This is a seasonal signal: water present or absent by date, keyed by a per-water-body id.

What we have	Count
Total polygons (WGS84)	876
water	720
non-water	156
Spatial spread (0.25° grid)	0.61

Deploying this on a server

- Runtime: a Python virtual environment from the requirements file, served with uvicorn. State is plain files (the hierarchy, the cards, the model binaries), so there is no database to run.
- Earth Engine: the app initialises with a project id and the runtime user's Earth Engine credentials, so on a server we authenticate once as that user. A service-account key is the natural choice for a fully headless box.
- Publishing to GitHub: the model zoo is its own git repository pushed to a configured remote. We leave authentication to the system git, so a server needs either a token through a credential helper or an SSH deploy key.
- Configuration is a small set of environment variables: the Earth-Engine project and user, and the zoo remote. Committed model binaries let a fresh clone classify straight away.

Thank you